Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Lecture 3

Graph

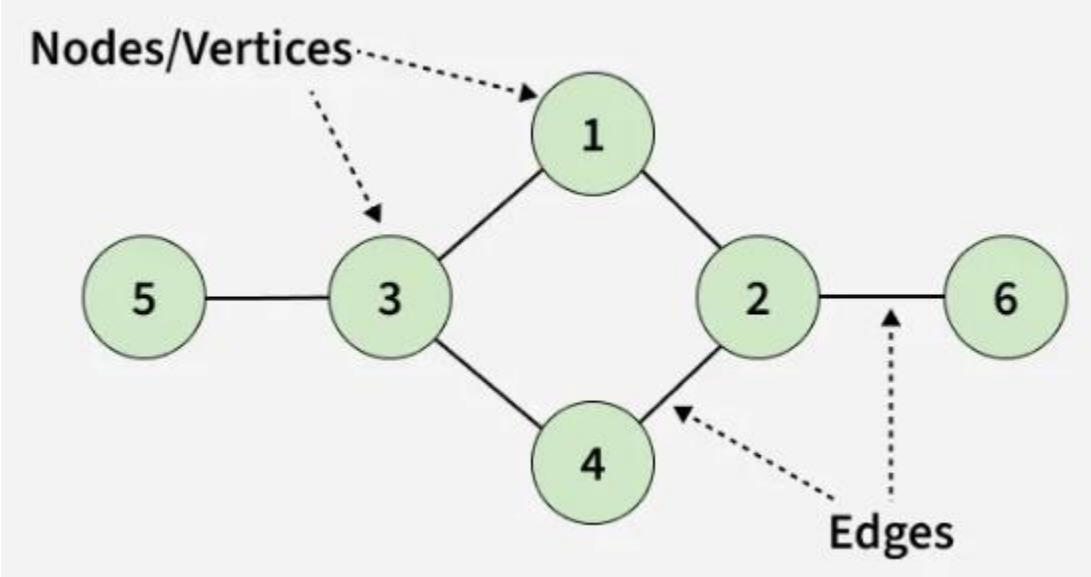
Dr Rahaf Ghazal

2025-2026

Introduction

A graph is a nonlinear data structure consisting of **vertices** and **edges**.

Example: On a map, each city is a vertex, and each road connecting two cities is an edge. This way, a graph represents how cities are linked.



So, we can say that the formal definition of Graph :

$$G=(V,E)$$

Where:

V: a finite, non-empty set of vertices.

E: a set of edges

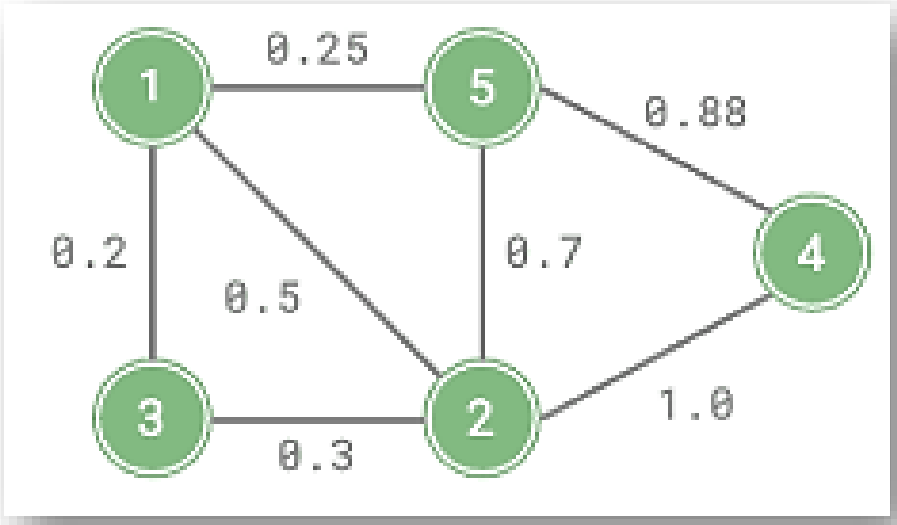
Types Of Graphs in Data Structure and Algorithm

A graph can be divided into multiple categories based on different properties such as edges, direction, connectivity and many others.

Based on weight :

1. Weighted Graphs

is a graph where each edge has a number (weight) that represents distance, cost, or time.

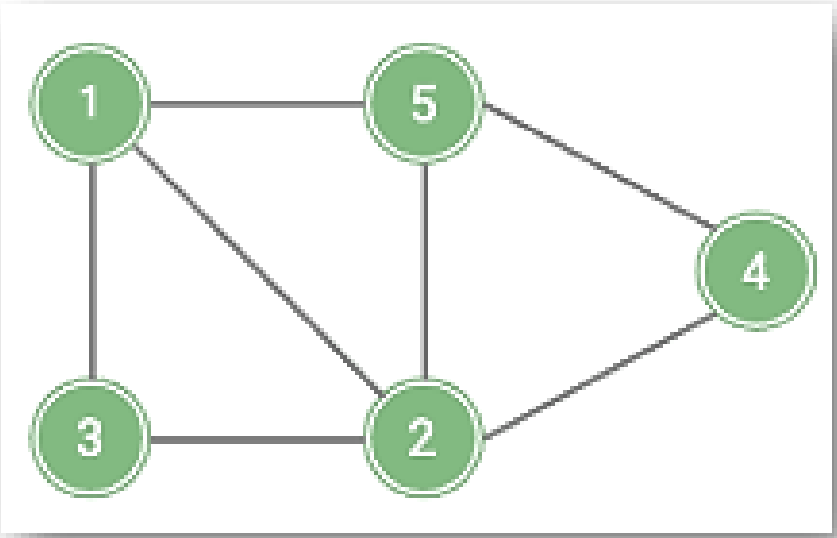


Weighted graph

(Edges have weight attributes)

2.Unweighted Graphs

is a graph where all edges are treated equally, with no extra values like distance or cost.



Unweighted graph

(All edges are equivalent)

Common examples on real-world weighted networks:

- ✓ **Google Maps Navigation (Distance/Time):** تُربط المواقع (العُقد) بالطرق (الأضلاع)، وتمثل الأوزان مسافة القيادة (بالأميال) أو وقت السفر المُقدّر (بالدقائق).
- ✓ **Computer Networks (Bandwidth/Latency):** مراكز البيانات هي العقد، والكابلات هي الأضلاع. تمثل الأوزان زمن الاستجابة أو النطاق الترددي المتاح.
- ✓ **Social Networks (Connection Strength):** يُمثل المستخدمون العُقد، والعلاقات تُمثل الأضلاع. يمكن أن تُمثل الأوزان معدل التفاعل (مثل عدد الرسائل المُرسلة).

غالبًا ما تهدف هذه التطبيقات إلى حل "مشكلة أقصر مسار" لإيجاد توليفة من أقل عدد من الروابط وأقل وزن إجمالي.

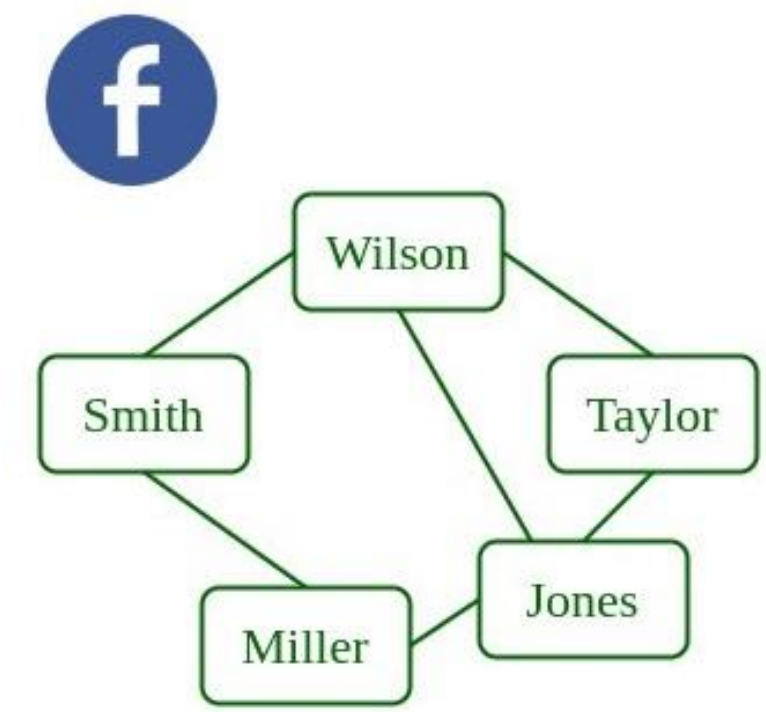
Based on Edge Direction:

1. Undirected Graphs

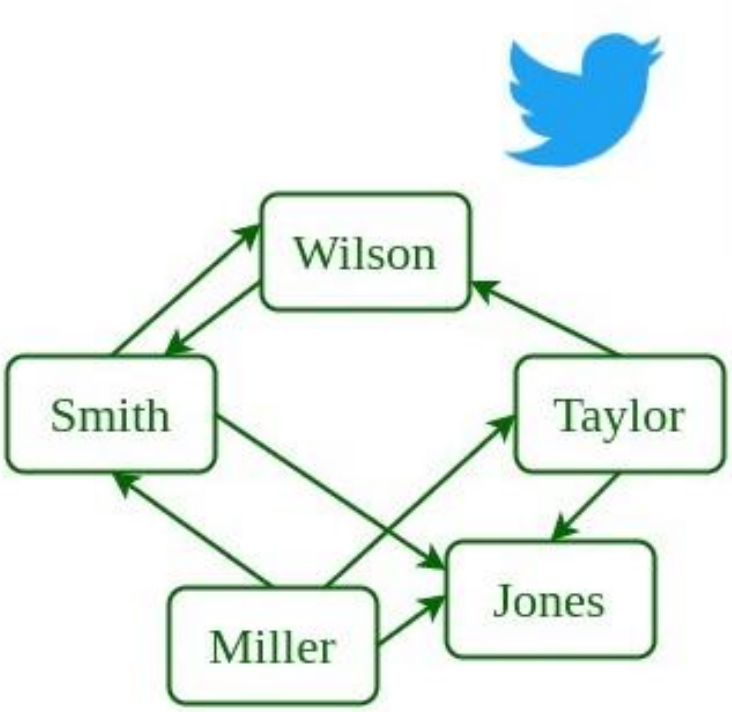
In undirected graphs, edges represent a “two - way" connection between two vertices.

2.Directed Graphs

In directed graphs, edges have directionality, meaning edges A->B and A <-B are independent of each other.



Undirected Graph

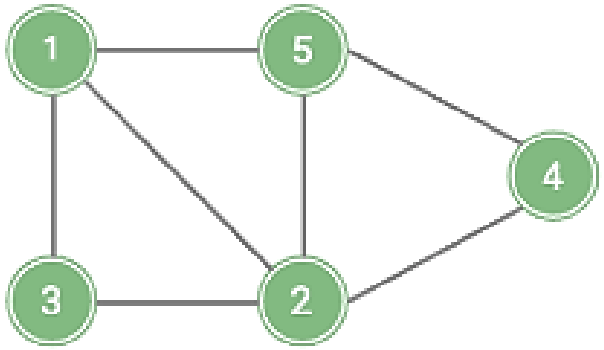


Directed Graph

Based on Edge connected:

1. Connected Graphs

For connected graphs, starting from any vertex, all other vertices can be reached.

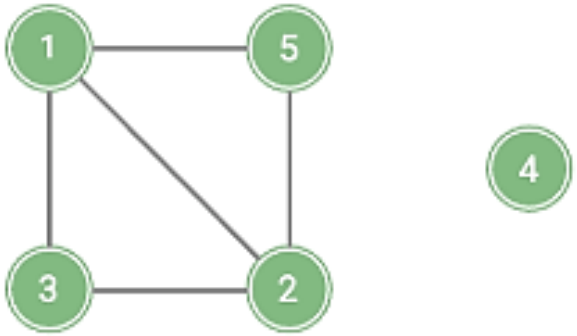


Connected graph

(All vertices are reachable)

2.Disconncted Graphs

For disconnected graphs, starting from a certain vertex, at least one vertex cannot be reached.

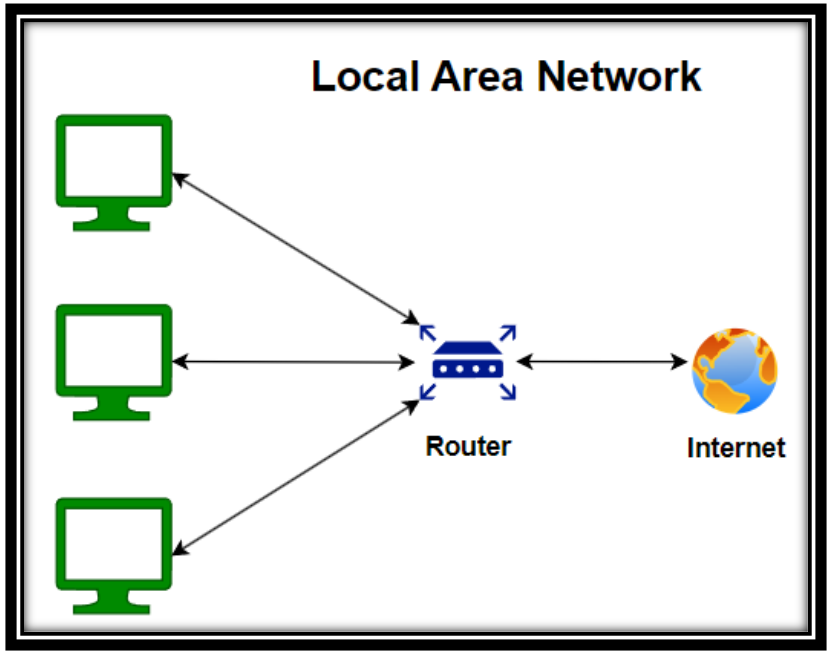


Disconnected graph

(Some vertices are unreachable)

Common examples on real-world connected networks:

- ✓ **Internet/Web Graph:** ممكن أن نرى بأن صفحات الانترنيت (العقد) تتصل مع بعضها بروابط تشعبية (الأضلاع) وعادة مايكون هذا البيان متصل تماماً، مما يسمح بالتنقل من صفحة إلى أخرى.
- ✓ **Road Networks:** بفرض لدينا خريطة لنظام الطرق السريعة في دولة ما، حيث تمثل المدن العقد والطرق الأضلاع. عادةً ما تكون الخريطة (البيان) متصلة.
- ✓ **Computer Networks:** A LAN (Local Area Network) where every computer is connected to a central switch.



Common examples on real-world Disconnected networks:

- ✓ **Island Nations:** بفرض لدينا خريطة جزر حيث تمثل العقد المدن، عندها يمكن أن تكون هذه الخريطة(البيان) غير متصل بحال عدم وجود جسور أو أنفاق تربط بعض الجزر.
- ✓ **Disjoint Social Groups:** مجموعتان منفصلتان من الأشخاص على منصة تواصل اجتماعي، لا تربطهما أي صداقات أو علاقات مشتركة.
- ✓ **Disconnected IT Infrastructure:** شبكة شركة لا يوجد فيها اتصال مادي أو منطقي بين شبكة الموظفين وشبكة قاعدة البيانات الآمنة.

Graph Terminology

Graph data structures include the following commonly used terms:

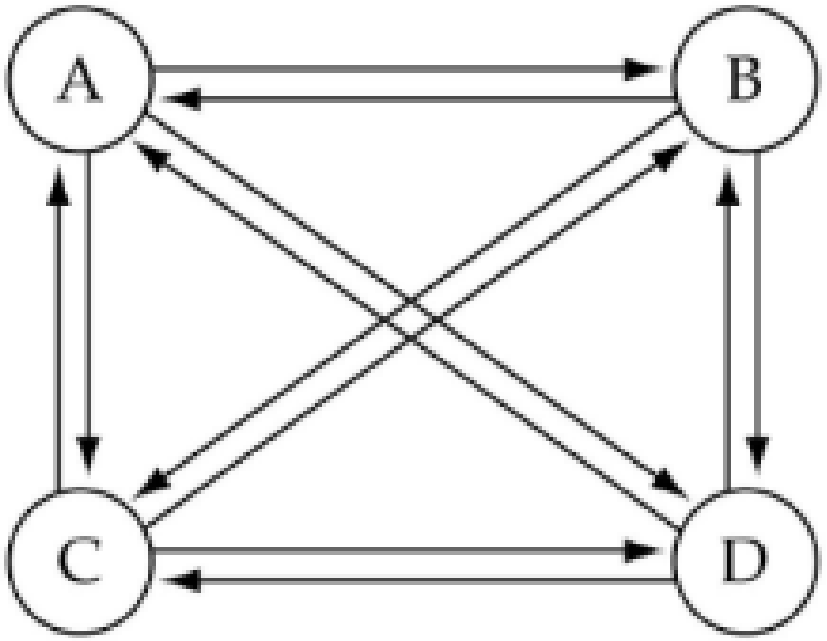
- **Adjacency:** When two vertices are connected by an edge, these two vertices are said to be "adjacent".
- **Path:** The sequence of edges from vertex A to vertex B is called a "path" from A to B.
- **Path Length:** Number of edges on the path.
- **Path cost:** sum of weight of edges in a path.
- **Cycle:** is a path that begins and ends at the same node
- **Degree:** The number of edges a vertex has. For directed graphs, in-degree indicates how many edges point to the vertex, and out-degree indicates how many edges point out from the vertex.
- **Complete graph:** a graph in which every vertex is directly connected to every other vertex.

Note:

- What is the number of edges in a complete directed graph with N vertices?

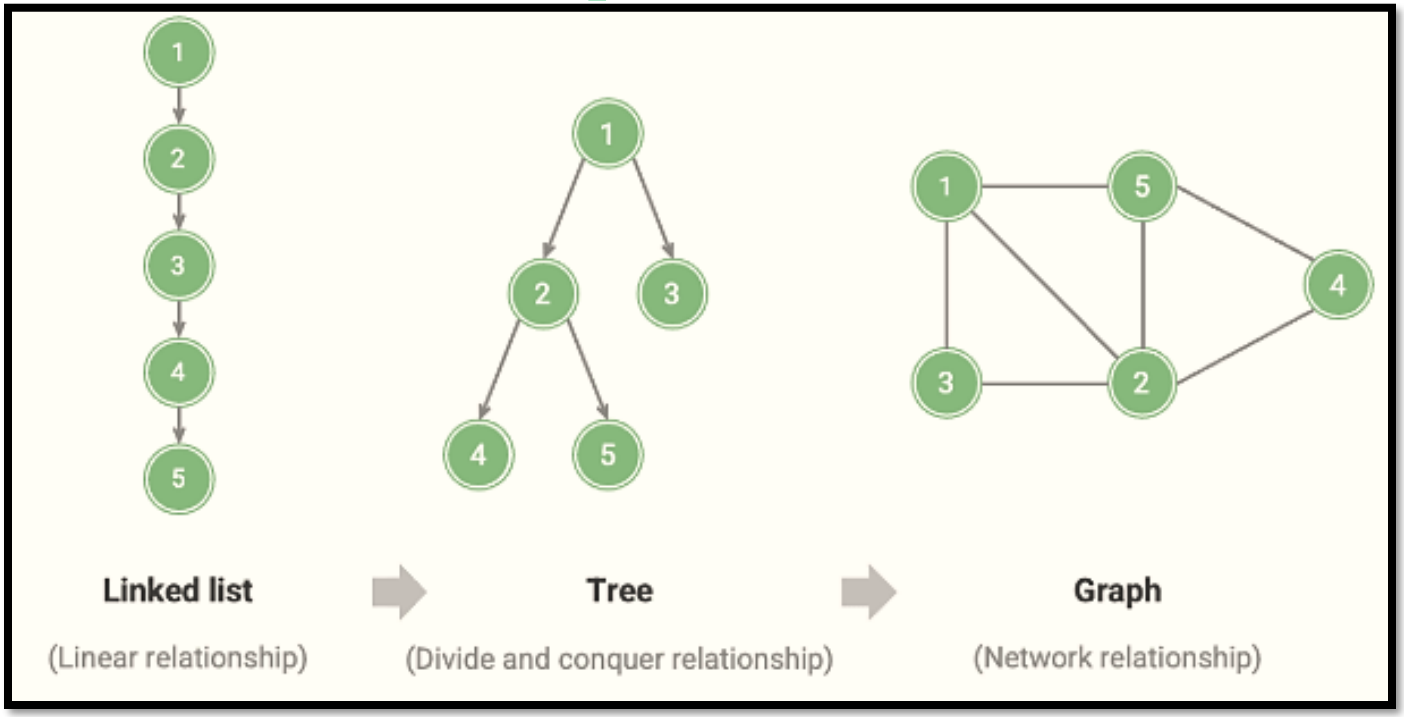
*N * (N-1)*

$O(N^2)$



Tree VS. Graph

Tree is a restricted type of Graph Data Structure, just with some more rules. Every tree will always be a graph but not all graphs will be trees. **Linked List**, **Trees**, and **Heaps** all are special cases of graphs.



Key Differences Between Graph and Tree:

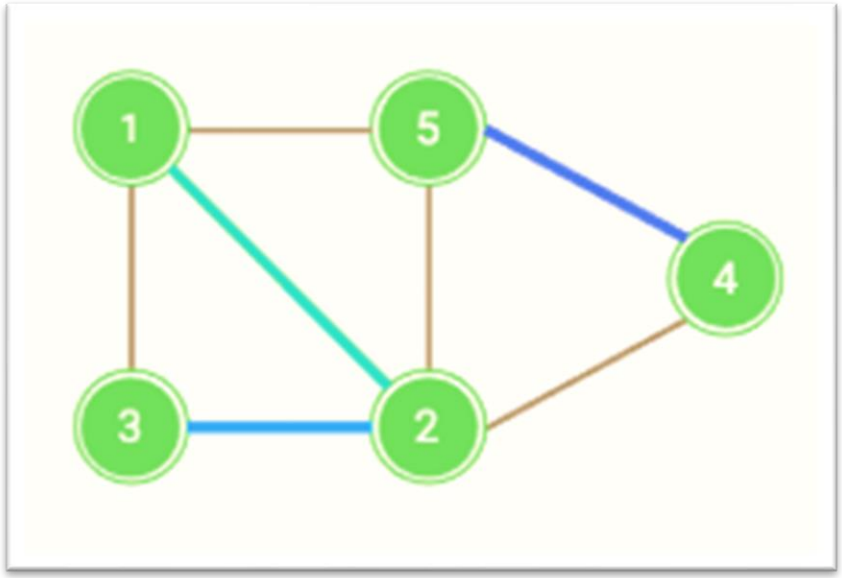
- **Cycles:** Graphs can contain cycles, while trees cannot.
- **Connectivity:** Graphs can be disconnected (i.e., have multiple components), while trees are always connected.
- **Hierarchy:** Trees have a hierarchical structure, with one vertex designated as the root. Graphs do not have this hierarchical structure.
- **Applications:** Graphs are used in a wide variety of applications, such as social networks, transportation networks, and computer science. Trees are often used in hierarchical data structures, such as file systems and XML documents.

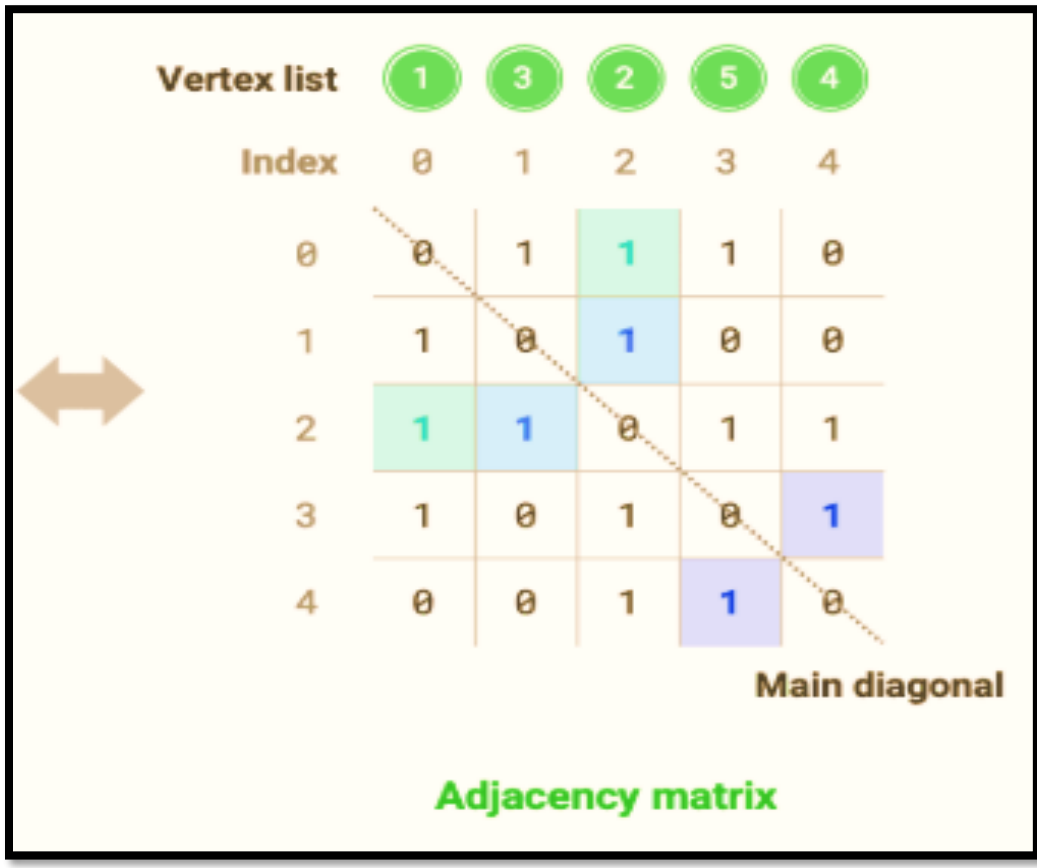
Representation of Graphs

Common representations of graphs include "**adjacency matrices**" and "**adjacency lists**". The following uses undirected graphs as examples.

1. Adjacency Matrix

In this method, the graph is stored in the form of the **2D matrix**, where each row (column) represents a vertex, and matrix elements represent edges, using **0** or **1** to indicate whether an edge exists between two vertices.





Adjacency matrices have the following properties:

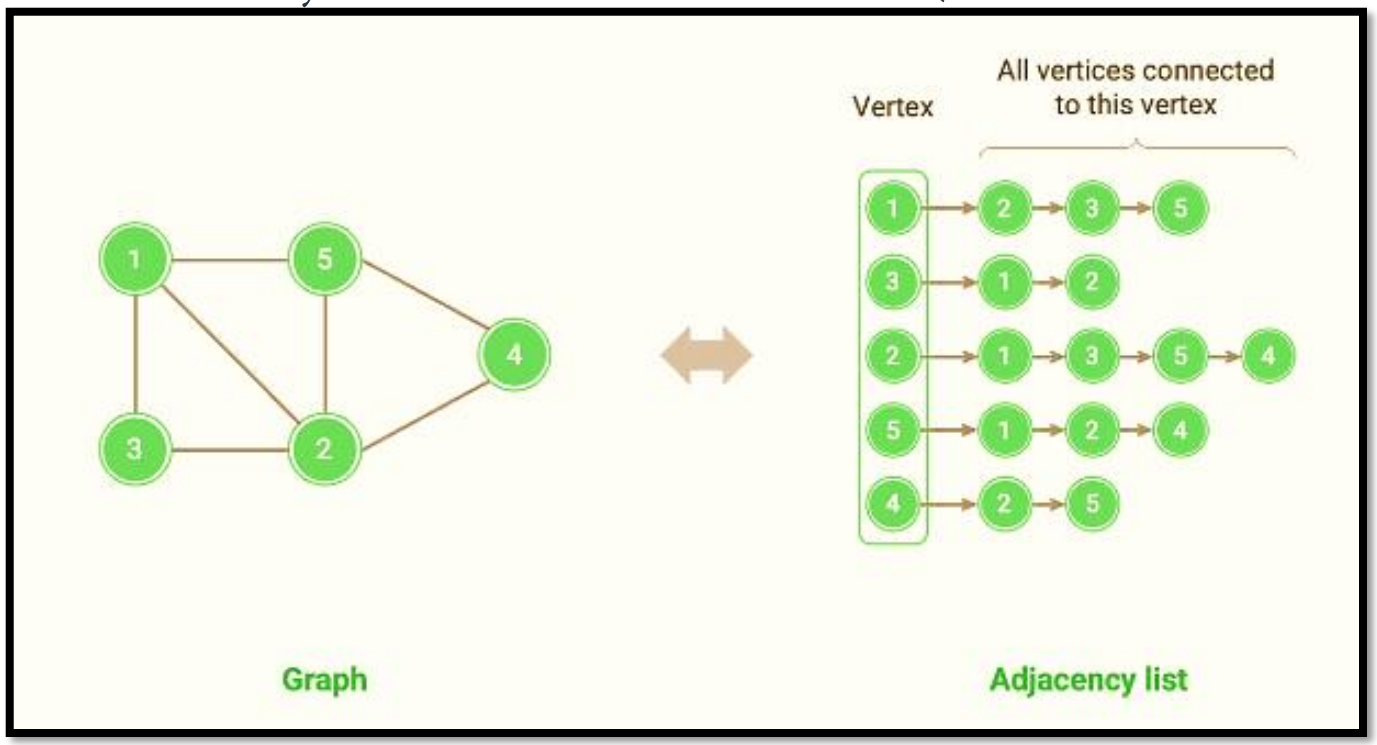
For reading

- In simple graphs, vertices cannot connect to themselves, so the elements on the main diagonal of the adjacency matrix have zero values.
- For undirected graphs, edges in both directions are equivalent, so the adjacency matrix is symmetric about the main diagonal.
- Replacing the elements of the adjacency matrix from 1 and 0 to weights allows representation of weighted graphs.
- When using adjacency matrices to represent graphs, we can directly access matrix elements to obtain edges, resulting in highly efficient addition, deletion, lookup, and modification operations, all with a time complexity of O(1) . However, the space complexity of the matrix is $O(n^2)$, which consumes significant memory.

Representation of Graphs

2. Adjacency List

This graph is represented as a collection of array lists. Uses linked lists to represent a graph, with linked list nodes representing vertices. The i-th linked list corresponds to vertex **i** and stores all adjacent vertices of that vertex (vertices connected to that vertex).




```
const n=100;
struct vertex{
        element val;    //  element= int , float , char , ...
        vertex* next;
};
typedef vertex* LGraph[n];
```

Adjacency Lists have the following properties:

For reading

- Adjacency lists only store edges that actually exist, and the total number of edges is typically much less than n^2, making them more space-efficient. However, finding edges in an adjacency list requires traversing the linked list, so its time efficiency is inferior to that of adjacency matrices.
- Observing previous Figure, the structure of adjacency lists is very similar to "chaining" in hash tables, so we can adopt similar methods to optimize efficiency. For example, when linked lists are long, they can be converted to AVL trees or red-black trees, thereby optimizing time efficiency from O(n) to O(log(n))
- linked lists can also be converted to hash tables, thereby reducing time complexity to O(1).

Graph Traversal

Traversal techniques are used to visit all the vertices of a graph systematically. These methods help to explore the graph completely and are useful for solving many graph-related problems.

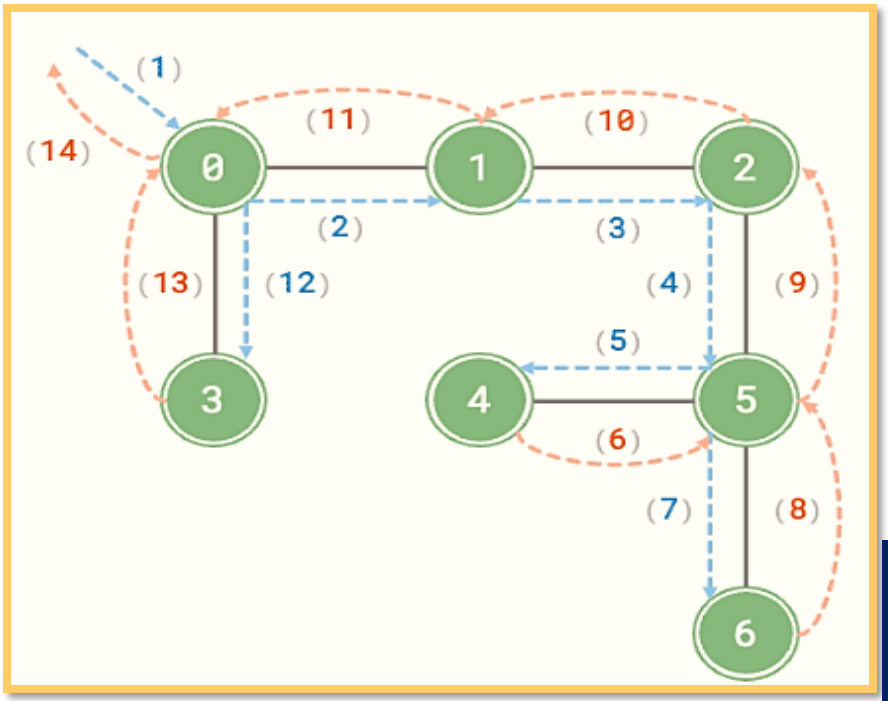
Depth First Search (DFS):

- Explores as far as possible along each branch before backtracking.
- Uses a stack (or recursion).

Breadth First Search (BFS)

- Explores all neighbors of a vertex before moving to the next level.
- Uses a queue.

Depth-First Search or DFS for a Graph



Graph's depth-first traversal (DFS)

Starting from vertex 0

Traversal sequence is

0, 1, 2, 5, 4, 6, 3

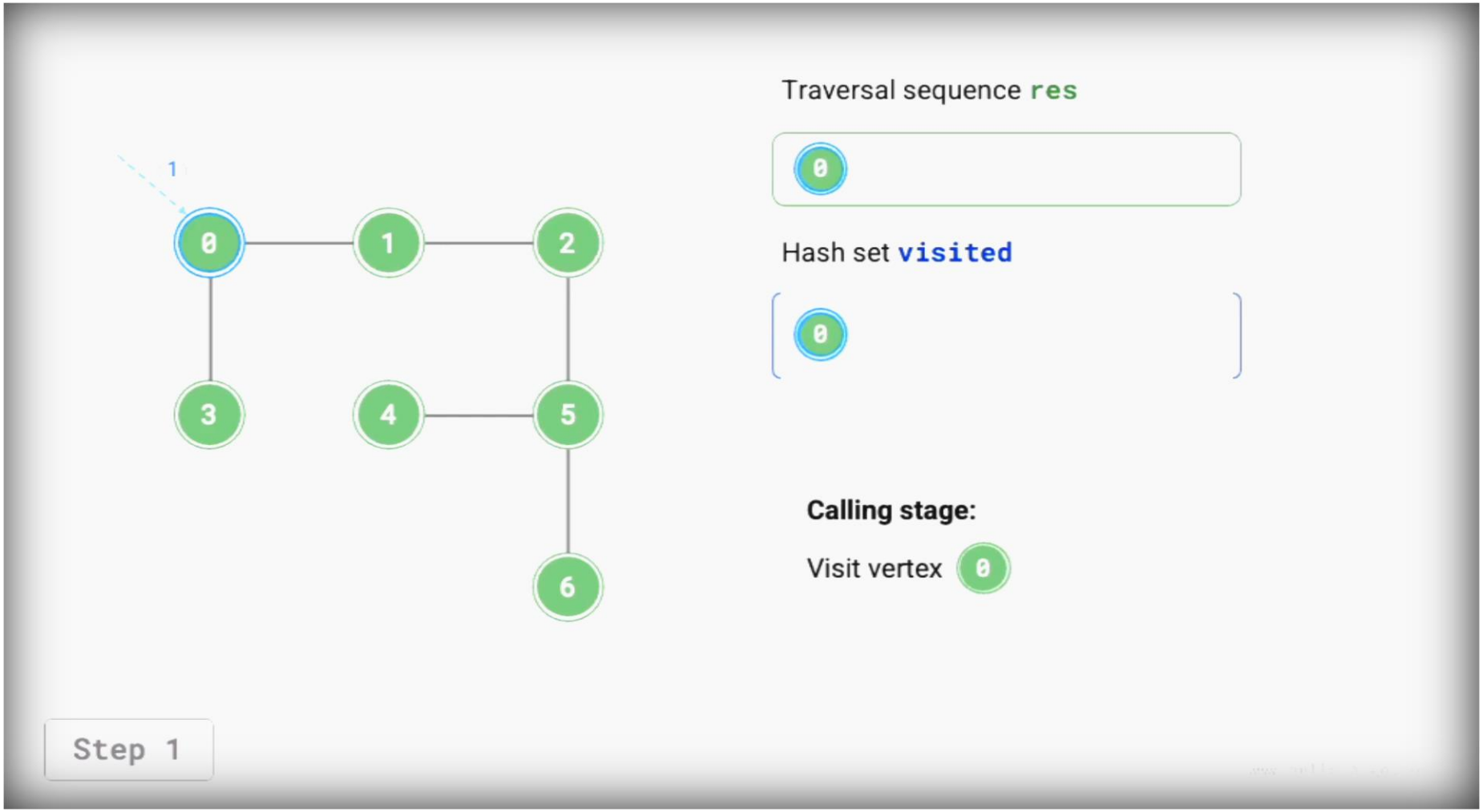
Algorithm Implementation:

This "go as far as possible then return" algorithm paradigm is typically implemented using recursion. We also need a hash set visited to record visited vertices and avoid revisiting

Pseudocode:

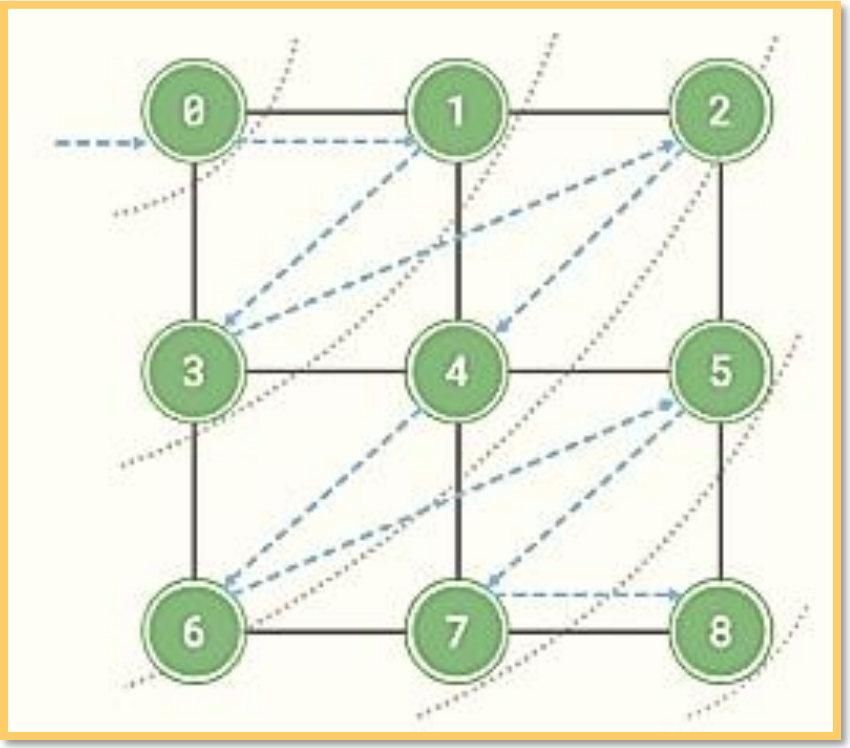
```
bool arr[ ][ ], visited[n] = { false } ;
Void DFS (Integer node)
        Print (node);
        Visited [node] = true ;
        for (i = 1 to n)
                if (arr [node][i] == true)
                        if (not visited [i])
                                DFS (i);
```

Video execution



Time complexity: O(V + E). We visit every vertex at most once and every edge is traversed at most once (in directed) and twice in undirected.

Breadth-First Search or BFS for a Graph



Graph's breadth-first traversal (BFS)

Starting from vertex 0 ,

Traversal sequence is

0, 1, 3, 2, 4, 6, 5, 7, 8

Algorithm Implementation:

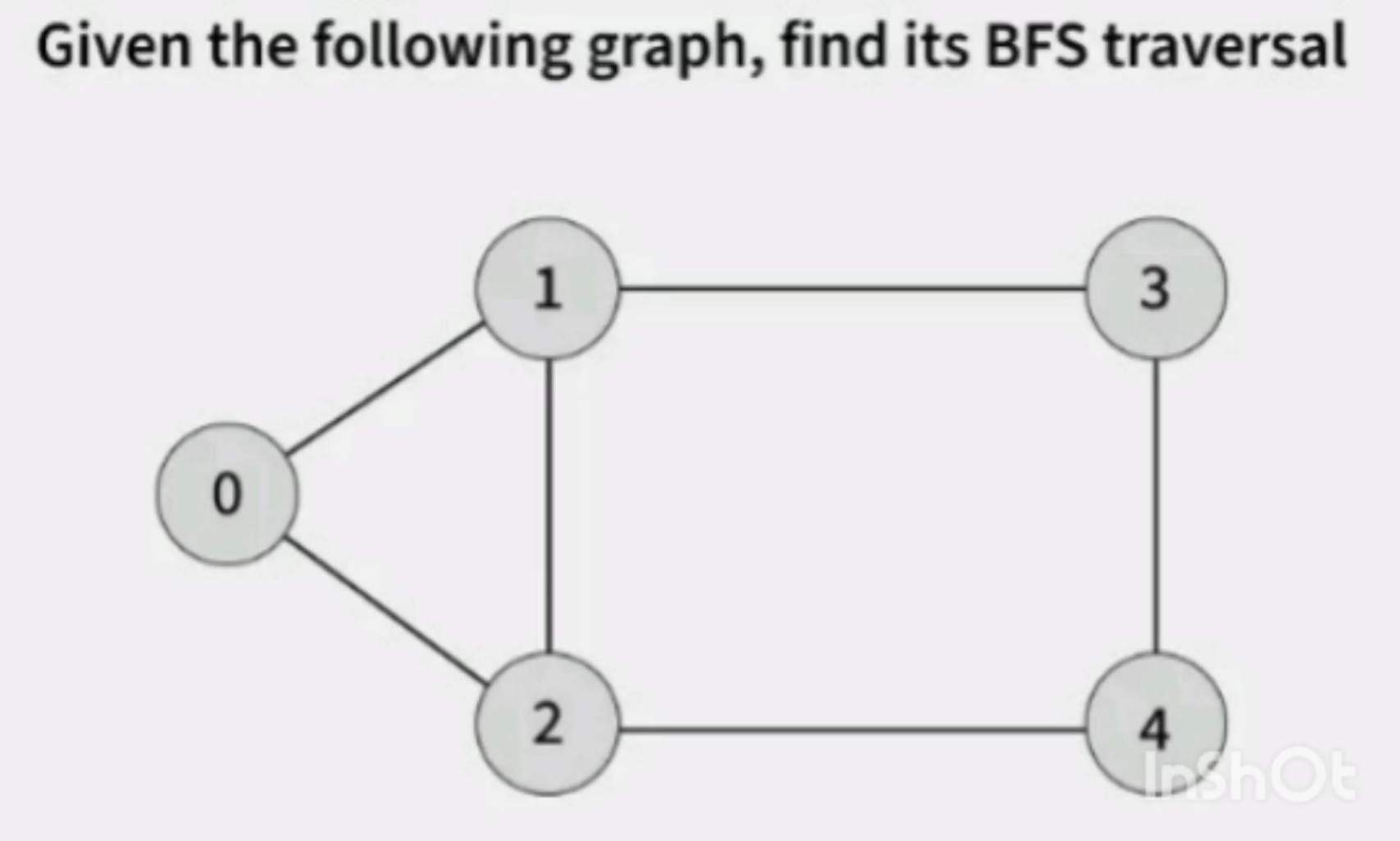
BFS is typically implemented with the help of a queue. The queue has a "first in, first out" property, which aligns with the BFS idea of "near to far".
To prevent revisiting vertices, we use a hash set visited to record which nodes have been visited.

Pseudocode:

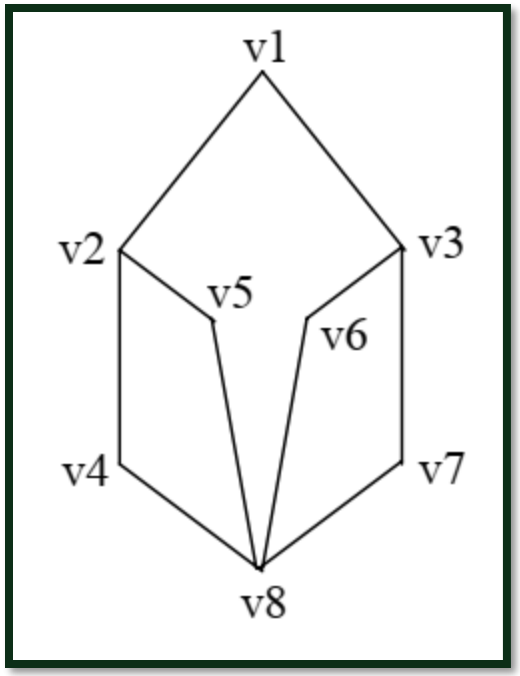
```
bool visited [n] = {false};
BFS(bool G[][] , Node s)
        Queue Q = ∅
        Q.Enqueu (s)
        visited[s] = true
        while (not Q.Empty( ) )
                current = Q.Dequeue( )
                print (current);
                for (i = 1 to n)
                        if (G[current][i])
                                if (not visited[i])
                                        Q . Enqueue (i)
                                        visited [i] = true
```

نخزن في الـ Queue أول عقدة كي لا يتحقق شرط التوقف في البداية

Video execution



Time complexity: O(V + E), BFS explores all the vertices and edges in the graph. It visits every vertex and edge only once.



مثال : ليكن لدينا البيان غير الموجه التالي و لتكن v1 عقدة البداية :

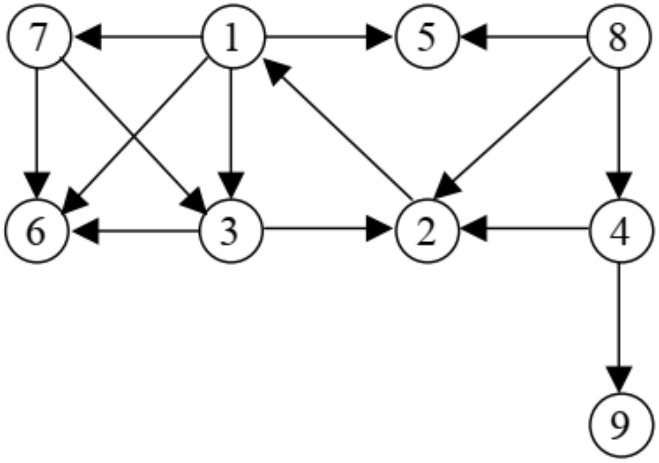
نتيجة مسح البيان بالعمق أولاً (=الطواف بالعمق) :

v1,v2,v4,v8,v5,v6,v3,v7

نتيجة مسح البيان بالعرض أولاً (=الطواف بالعرض) :

v1,v2,v3,v4,v5,v6,v7,v8

مثال:



عند تطبيق خوارزمية التجول بالعمق أولاً على البيان G الموضح في الشكل المجاور, فإن ترتيب المرور في العقد هو:

1 , 3 , 2 , 6 , 5 , 7 ,
4 , 9 ,
8

أما بالنسبة للتجول بالعرض أولاً يكون ترتيب المرور :

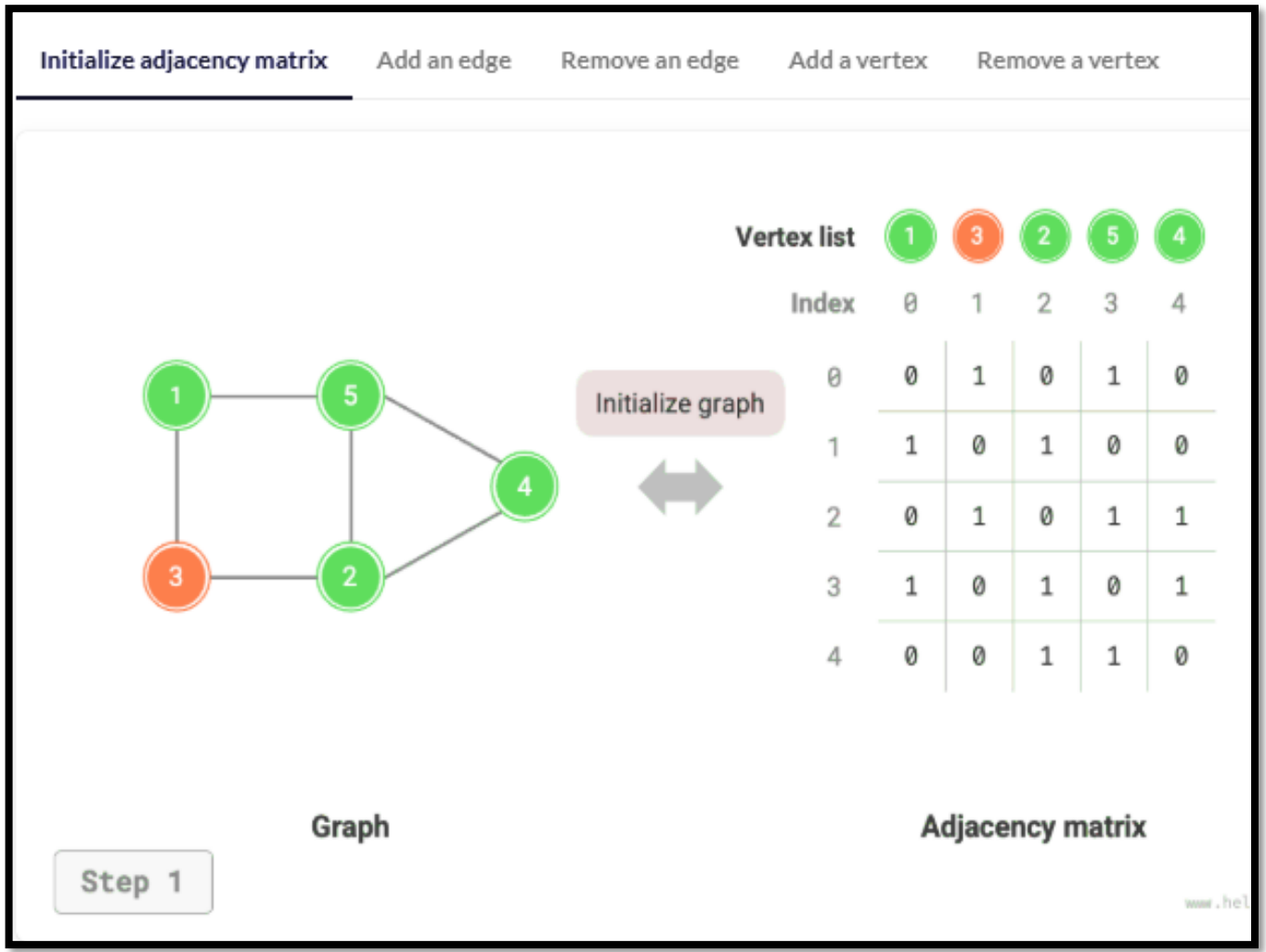
1 , 3 , 5 , 6 , 7 ,
2 ,
4 , 9 ,
8

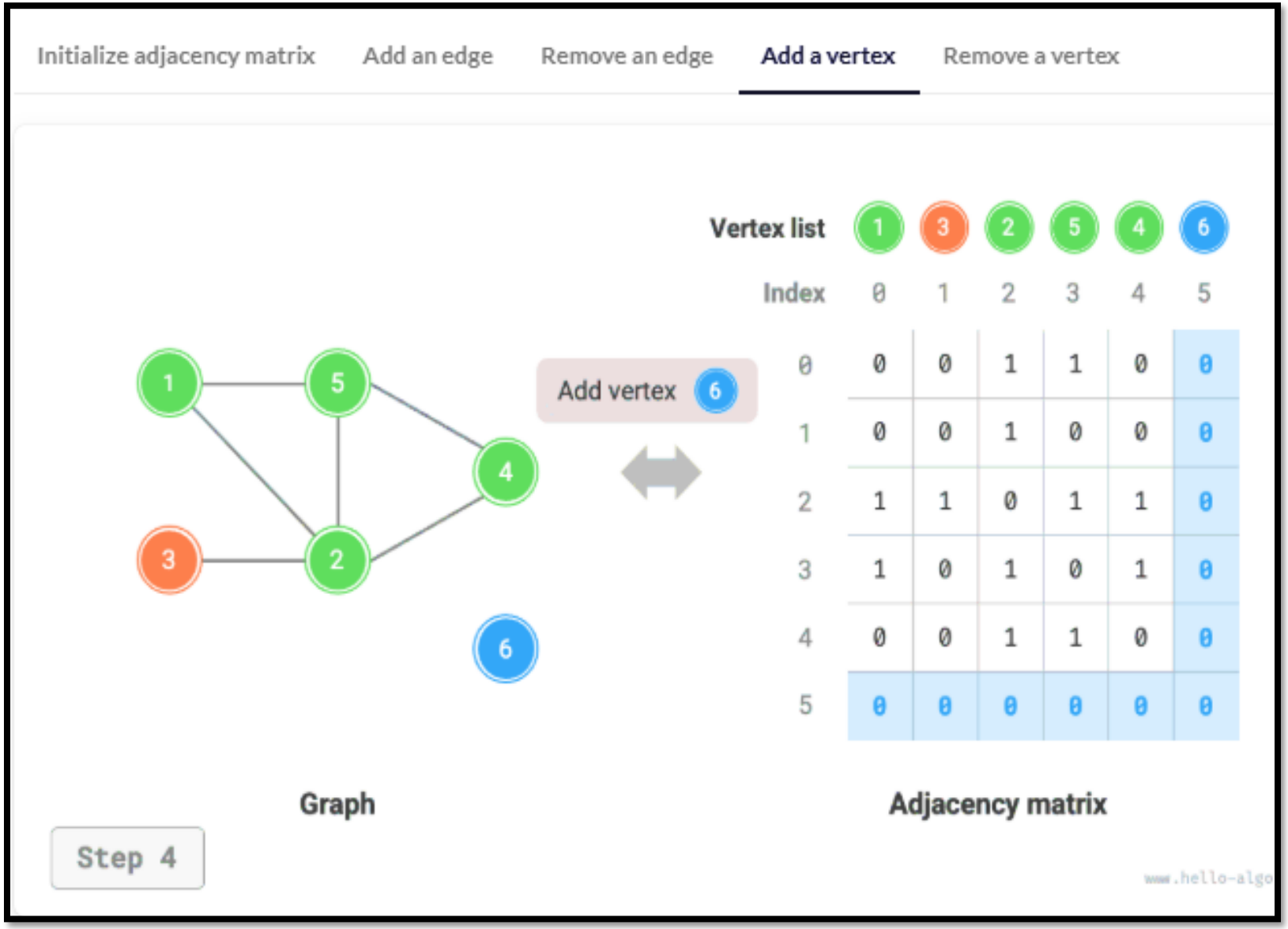
Basic Operations on Graphs

Basic operations on graphs can be divided into operations on "edges" and operations on "vertices". Under the two representation methods of "adjacency matrix" and "adjacency list", the implementation methods differ.

Implementation Based on Adjacency Matrix

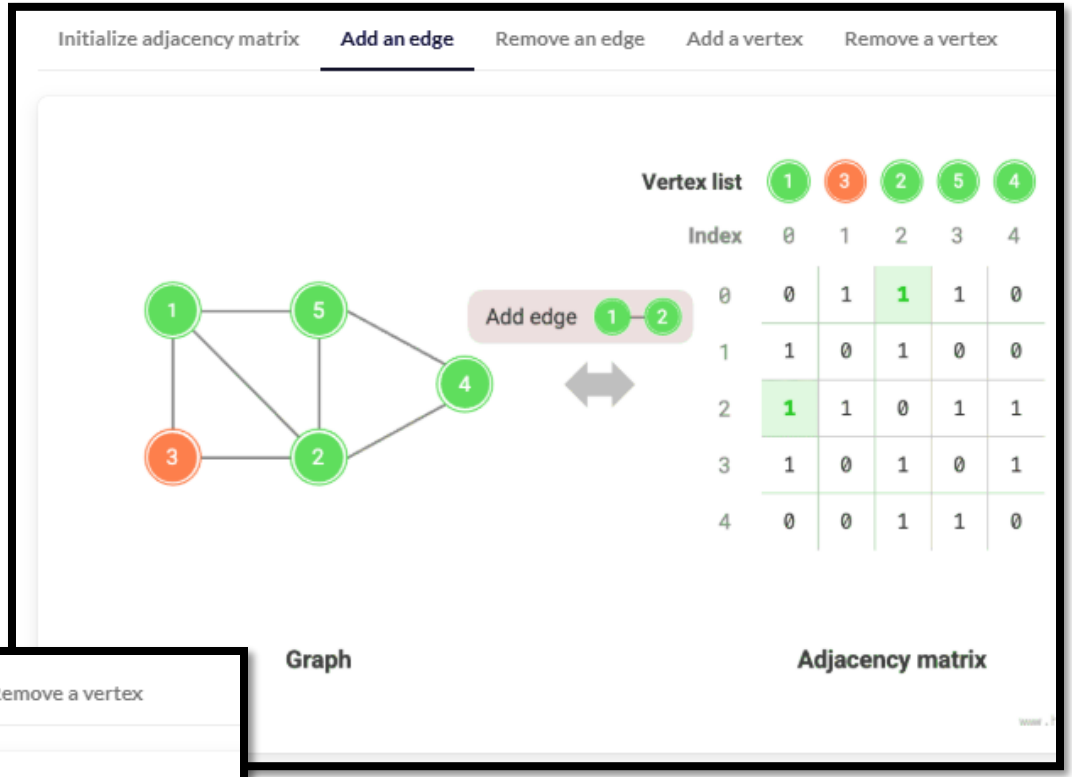
Initialization:; initialize an adjacency matrix adjMat of size nxn , using $O(n^2)$ time.

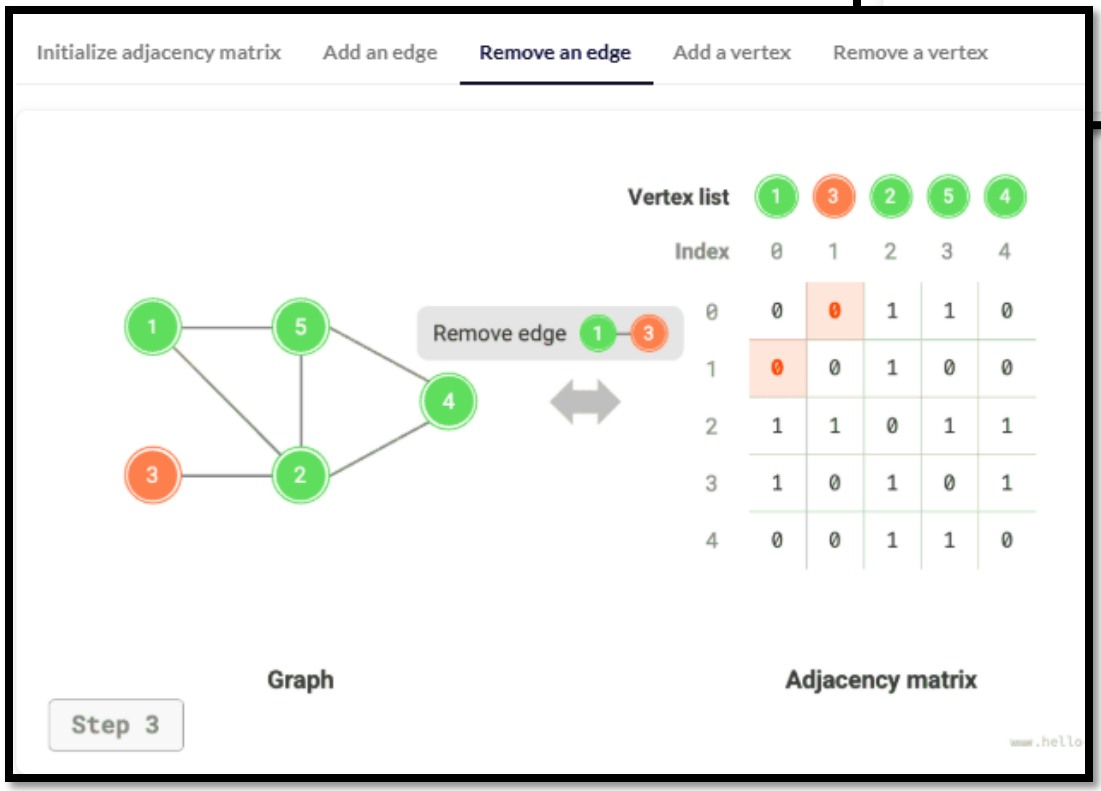


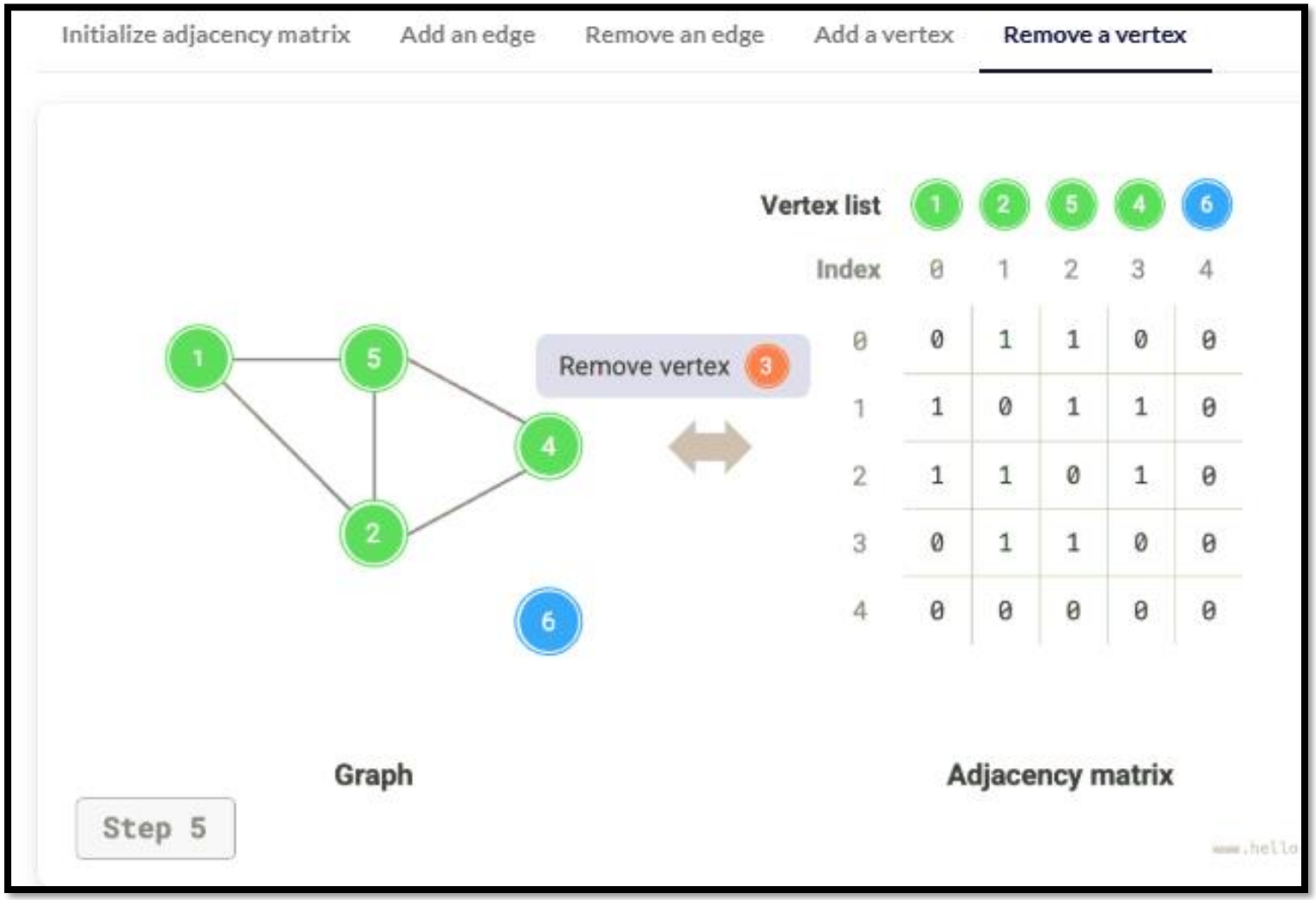


Adding a vertex: Add a row and a column at the end of the adjacency matrix and fill them all with 0 s, using $O(n)$ time.

Adding or removing an edge: Directly modify the specified edge in the adjacency matrix, using **O(1)** time. Since it is an undirected graph, both directions of the edge need to be updated simultaneously.

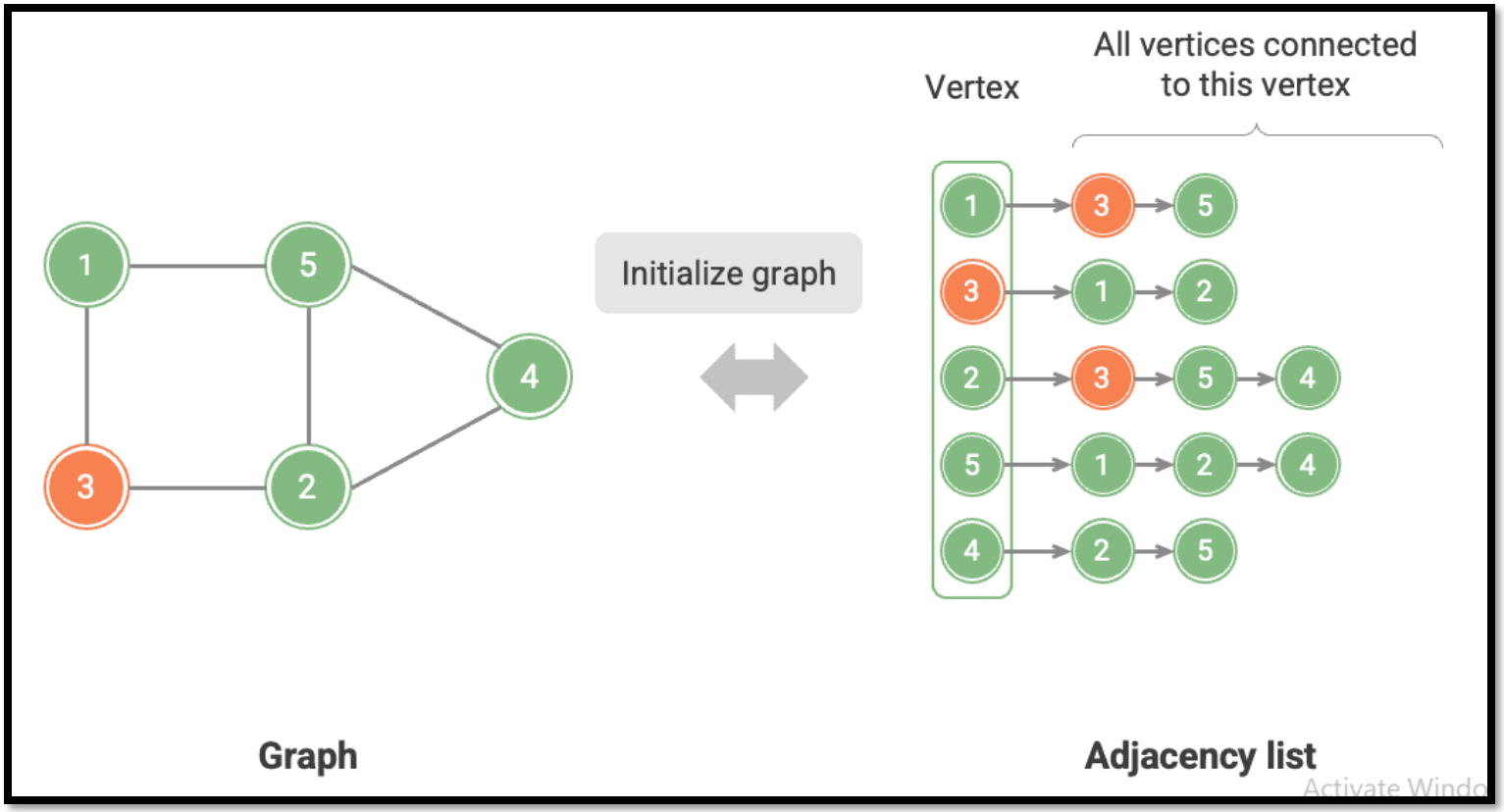




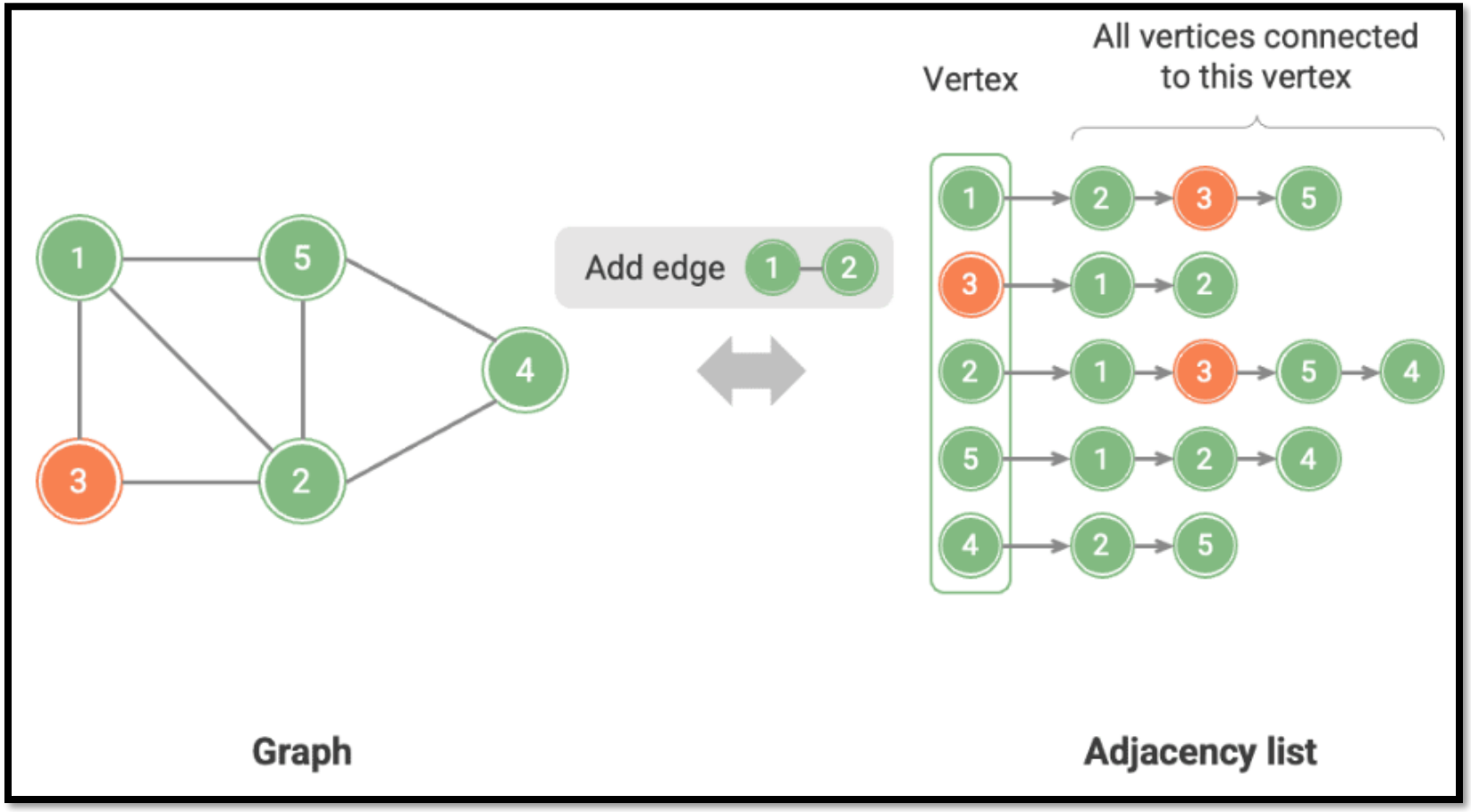


Removing a vertex: Delete a row and a column in the adjacency matrix. The worst case occurs when removing the first row and column, requiring $(n-1)^2$ elements to be "moved up and to the left", thus using $O(n^2)$ time..

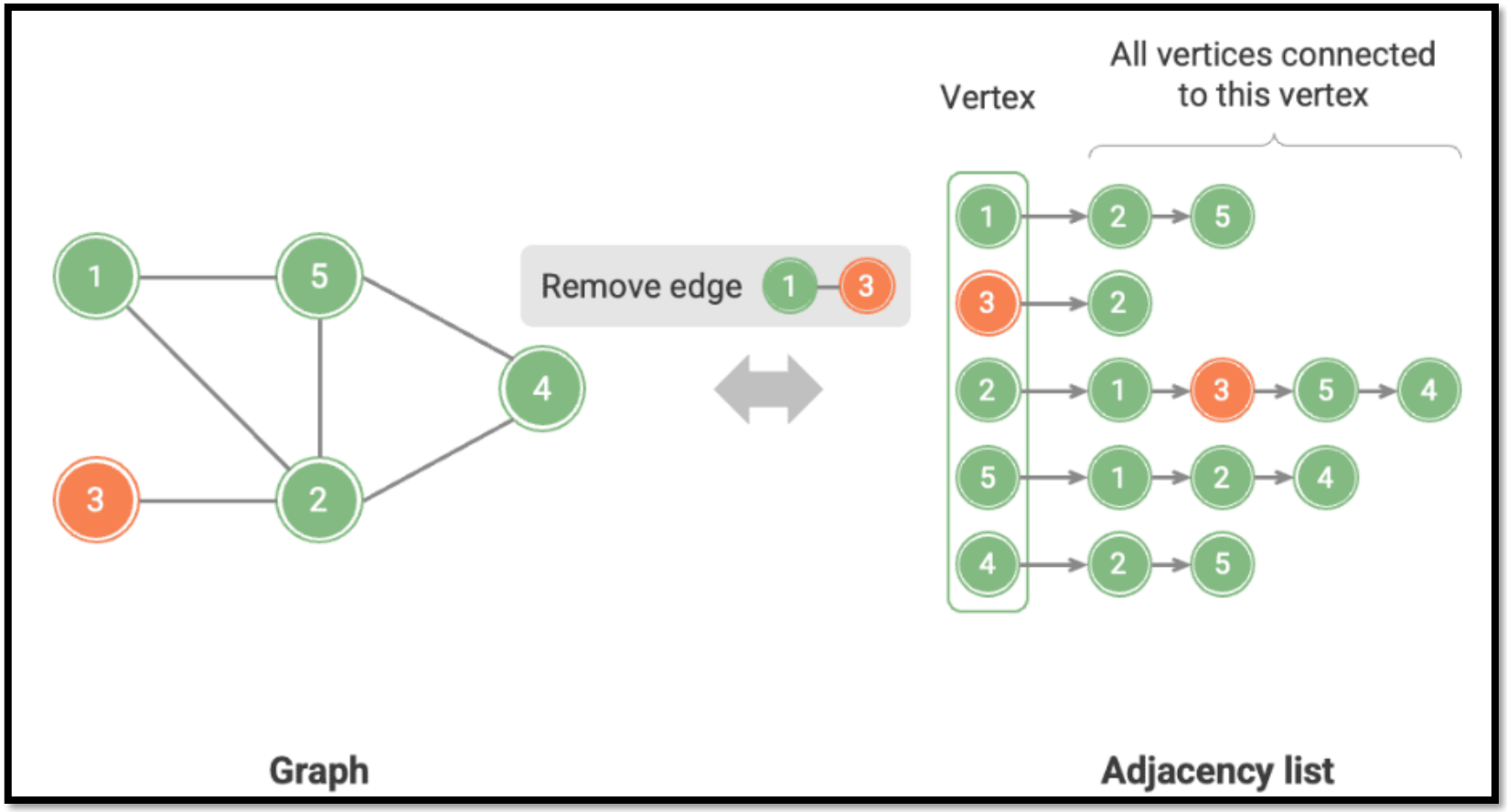
Implementation Based on Adjacency List



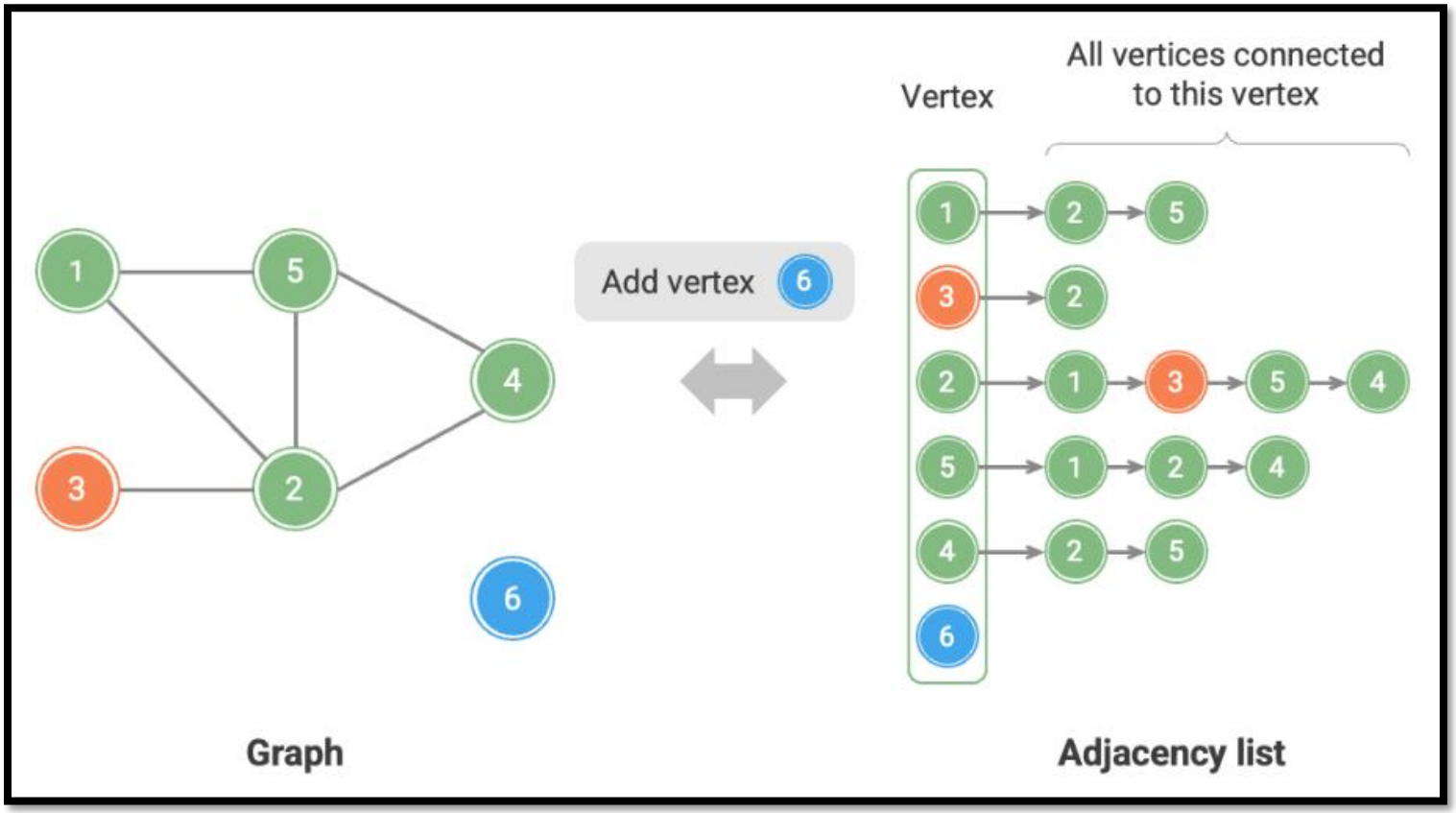
Initialization: Create n vertices and m edges in the adjacency list, using $O(n+m)$ time.



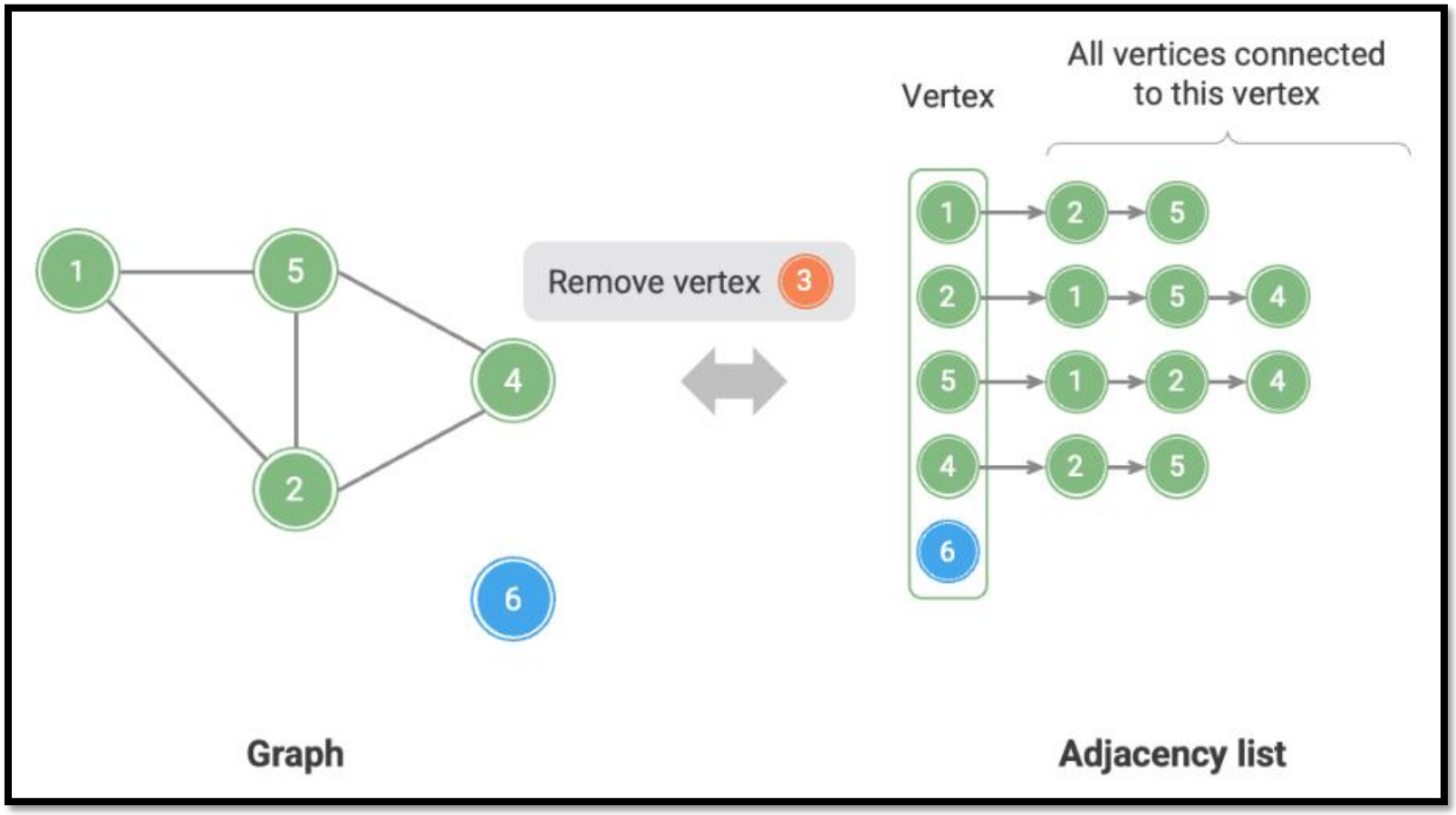
Adding an edge: Add the edge at the end of the corresponding vertex's linked list, using *O(1)* time. Since it is an undirected graph, edges in both directions need to be added simultaneously.



Removing an edge: Find and remove the specified edge in the corresponding vertex's linked list, using $O(m)$ time. In an undirected graph, edges in both directions need to be removed simultaneously.



Adding a vertex: Add a linked list in the adjacency list and set the new vertex as the head node of the list, using *O(1)* time.



Removing a vertex: Traverse the entire adjacency list and remove all edges containing the specified vertex, using $O(n+m)$ time.